

Kairon → ERPNext

CRM Provisioning & Webhook Integration

API contracts, request bodies, MongoDB schema, the defects found and fixed during live verification, and the evidence from real end-to-end tests against a running Kairon + Frappe/ERPNext stack.

Scope: kairon/crm/* (Kairon repo) and apps/kairon_connector (Frappe app)

Verified against: Frappe 16.26.3 / ERPNext 16.27.0 / Frappe CRM 1.80.0, Python 3.14 runtime

Test tenants: Kairon Demo Restaurant (Tier 1, CRM-only), Kairon Tier2 Regression Check (Tier 2, ERPNext/POS)

Session date: 2026-09-13

Contents

1. Architecture overview

2. Defects found & fixed this session

3. Kairon public REST API — `/api/bot/{bot}/crm/*`

4. kairon_connector inbound webhook gateway API

5. Outbound ERPNext APIs Kairon calls internally

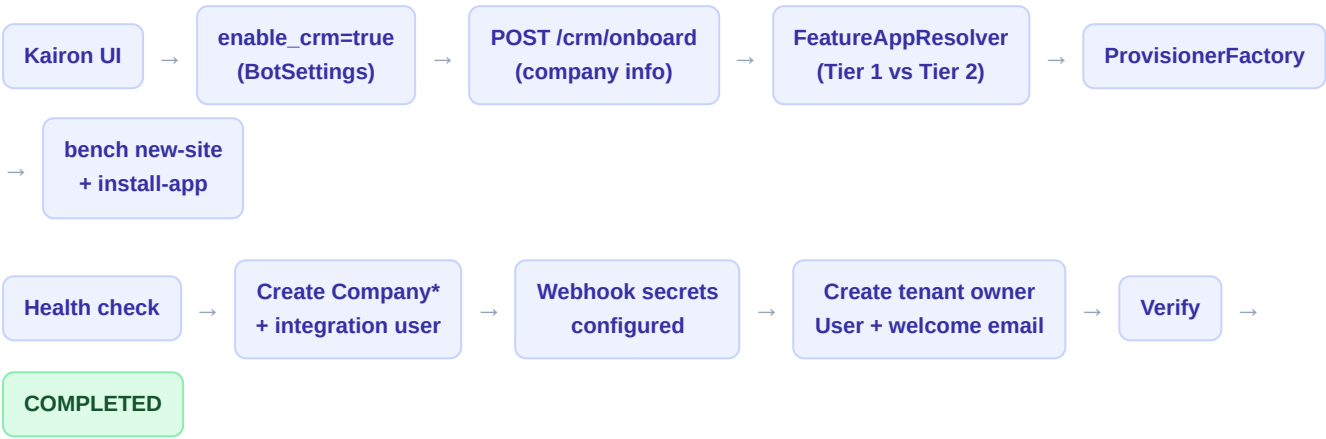
6. MongoDB collections & schema

7. Live verification evidence

8. Known limitations / follow-ups

1. Architecture overview

A Kairon bot admin enables CRM on a bot (`enable_crm=true`), then submits company details. Kairon provisions a dedicated Frappe site, installs the right tier of apps, wires up a bidirectional webhook relationship with that site, and hands the tenant owner a real login via email.



*Company (ERPNext DocType) only exists on Tier 2 sites. Tier 1 (standalone crm app) uses CRM Organization instead — see §2.

Tier resolution

TIER	TRIGGER	APPS INSTALLED	EXAMPLE
1 — Standalone	Only lightweight features selected (e.g. crm, helpdesk)	frappe, crm, kairon_connector	Company selects only "CRM"
2 — Full ERP Suite	Any monolith-dependent feature selected (POS, HR, Selling, Accounts...)	frappe, erpnext[, hrms], kairon_connector	Company selects "POS"

The two independent webhook relationships

DIRECTION	PURPOSE	SECURED BY	STORED IN
ERPNext → Kairon	User Invitation accepted → assign company permission	webhook_secret (HMAC-SHA256)	CRMClientDetails.webhook_secret (encrypted)
Kairon → ERPNext (kairon_connector)	Bot events (lead qualified, conversation message) → CRM Lead sync	Kairon Settings.webhook_secret (HMAC-SHA256, dual-secret rotation)	CRMClientDetails.lead_webhook_secret (encrypted)

2. Defects found & fixed this session

Before this session, the onboarding pipeline had never reached COMPLETED through its own real API path since a refactor introduced the first bug below (historical MongoDB records confirm every recent attempt was stuck at SITE_CREATED / FAILED_COMPANY / FAILED_USER). Nine defects were found and fixed, all verified live. All fixes to the Tier 2 (ERPNext) code path are additive/tier-gated — existing Tier 2 behavior is preserved except where the defect itself was breaking Tier 2.

FIXED 1. BenchExecutor.provision_site() — missing imports (NameError, every attempt)

kairon/crm/services/bench_executor.py called PreFlightValidator and ProvisionerFactory without importing either — a guaranteed NameError on every single onboarding call. ~160 lines of unreachable dead code after an early return (leftover from before the Strategy/Factory refactor) were also removed.

FIXED 2. Webhook gateway fail-open on missing secret (security)

kairon_connector/api/v1/webhook.py fell back to the literal string "default_secret" as the HMAC key if get_password() raised — an attacker who knew this constant could forge valid signatures. Now fails closed (401) instead.

FIXED 3. kairon_connector install crash on Tier 1 sites (Postgres transaction poisoning)

services/custom_fields.py ran unguarded UPDATE "tabDesktop Icon" SET name='CRM'... statements that collided with a row the crm app itself already created, throwing a unique-constraint violation. Because the catch block never called frappe.db.rollback(), every subsequent SQL call in the same install hook then failed with "current transaction is aborted." Each rename is now idempotent (guarded with NOT EXISTS) and isolated in its own commit/rollback boundary.

FIXED 4. CRM workspace hidden on the tenants that need it most

setup/bootstrap.py's _configure_workspace_visibility() unconditionally hid the "CRM" workspace as part of its restaurant-POS setup — which ran on every tenant, including Tier 1 CRM-only ones. Now skipped when the site is crm-only (no erpnext).

FIXED 5. Restaurant-POS bootstrap crashed on Tier 1 (ERPNext-only DocType references)

setup_restaurant_workspace(), create_restaurant_custom_fields(), and create_restaurant_client_scripts() all reference ERPNext-only DocTypes (Item, Item Group, POS Invoice, Warehouse, Company) that don't exist without erpnext installed. Now gated behind "erpnext" in frappe.get_installed_apps().

FIXED 6. Fictional role names ("CRM Admin", "CRM User" don't exist)

provisioning_matrix.yaml's crm entry assigned roles that exist nowhere in Frappe or the standalone crm app, causing LinkValidationError on every Tier 1 role assignment. Replaced with the real roles (System Manager, Sales Manager, Sales User) that BaseProvisioner already uses elsewhere for the same purpose.

FIXED 7. Company creation always attempted, even where the DocType doesn't exist

ProvisioningService unconditionally called create_company() and the company-permission/verification checks — all of which 500 on Tier 1 sites (no erpnext → no Company DocType). Now skipped for Tier 1 across provisioning.py, erpnext_client.py, and provision_verifier.py.

FIXED 8. Tenant owner's credential issuance was duplicated, and the real path never ran

This was the direct cause of the original concern raised in this session. BaseProvisioner .setup_user_and_migrate() (run during bench provisioning) already created the tenant owner's User record and set its password to the ephemeral, in-memory-only Administrator password — before ProvisioningService's own, correct create_erpnext_user() step (which decides between a welcome-email link or a temporary password) ever ran. Since the user already existed by then, that step silently skipped — leaving the tenant owner with a password nobody was ever told, no welcome email, nothing. Fixed by removing credential issuance from the bench-side script entirely; it now only touches an already-existing user's roles. A second, smaller issue in the same area: the native Frappe welcome-email hook didn't reliably fire via the REST create path on this Frappe build, so create_erpnext_user() now also explicitly calls Frappe's own whitelisted reset_password method as a fallback.

FIXED — most severe 9. Tier 2 (ERPNext/POS/KOT) onboarding marked COMPLETED immediately after site creation

ERPNextProvisioner.provision() set onboarding_status = COMPLETED directly, right after bench new-site succeeded — before Company creation, the integration user, webhook secrets, module isolation (in ProvisioningService's copy of that step), **the tenant owner's actual User record, and the welcome email** ever ran. Every later step in execute_onboarding_workflow guards on the document's current status, so once it read back "COMPLETED" from Mongo, every remaining step silently no-opped. The API response still reported "status": "COMPLETED" with a valid site URL, but on the actual site only "Administrator" existed, with a password nobody was ever given — **no Tier 2 tenant owner has ever had a working login**. Live-verified: before the fix, a fresh Tier 2 site had zero Email Queue entries and no tenant-owner User record at all. Fixed by setting SITE_CREATED instead (matching CRMProvisioner's already-correct Tier 1 behavior), letting the same state machine that now works for Tier 1 drive Tier 2 through Company creation, user creation, welcome email, and verification too.

KNOWN, NOT FIXED Minor: duplicate emails on some runs

When the native Frappe welcome-email hook does fire, the explicit `reset_password` fallback (fix 8) still also fires and sends a second email with a different link — generating the second link invalidates the first. Both work; the tenant owner just gets two emails instead of one. Left as-is pending a decision on which mechanism to keep as primary.

KNOWN, NOT FIXED Resuming from certain failure states beyond a single request needs more work

The state machine correctly resumes within one continuous `execute_onboarding_workflow()` call (verified live: a retry-from-FAILED_USER resumed correctly after the fixes above). A true separate-request resume was not exercised beyond that scenario; some steps still rely on locally recomputed values rather than everything being reconstructable purely from the persisted document.

3. Kairon public REST API — /api/bot/{bot}/crm/*

All endpoints require an authenticated Kairon session (Authorization: Bearer <access_token>, obtained via POST /api/auth/login) with ADMIN_ACCESS scope for the target bot. Base path: /api/bot/{bot}/crm.

POST /onboard

Initiates (or resumes) automated ERPNext/CRM site provisioning for the bot.

```
{
  "company_name": "Kairon Demo Restaurant",
  "abbr": "KDR",
  "default_currency": "INR",
  "country": "India",
  "selected_modules": ["CRM"]           // optional; omit for all business modules
}
```

Response: { status, site_name, site_url, company, erpnext_user, authentication_method, temporary_password, error }

GET /available-modules

Live-discovers installed ERPNext business modules for the module picker (falls back to the static catalog if the site is offline). No request body.

GET /selected-modules

Returns the currently selected/active modules for the provisioned tenant. No request body.

POST /configure-modules

```
{ "selected_modules": ["CRM", "POS", "Selling"] }
```

POST /reconcile-modules

Detects and repairs configuration drift between Kairon's record and the live ERPNext Module Profile/workspace state. No request body.

POST /create-user

```
{ "email": "sales@company.com", "role": "CRM User" }
```

POST /invite-user

Sends a native ERPNext User Invitation (no manual user creation).

```
{
  "email": "newuser@company.com",
  "roles": ["Sales User"],
  "company_name": "Kairon Demo Restaurant" // optional; resolved from CRMClientDetails if
omitted
}
```

POST /webhook/invitation-accepted

Public (guest-accessible) receiver for ERPNext's native Webhook, fired when an invited user accepts. Not bearer-token authenticated — secured by HMAC instead.

HEADER	PURPOSE
X-Kairon-Site	Identifies which tenant's CRMClientDetails.webhook_secret to verify against
X-Frappe-Webhook-Signature	HMAC-SHA256 signature of the raw body

```
{
  "status": "Accepted",
  "email": "newuser@company.com",
  "app_name": "frappe"
}
```

GET /status

Returns { onboarding_status } only — for lightweight UI polling during provisioning.

GET /details

Full CRMClientDetails document for the bot (secrets decrypted server-side where applicable before returning). No request body.

GET /users

Lists all users registered on the tenant's ERPNext site, via the integration user's API key. No request body.

GET /tables / GET /tables/{table_name}

Debug/inspector endpoints listing the tenant's Postgres tables and row content (?limit= query param, default 50, max 500).

DELETE /project?company_name=...

Deletes the CRMClientDetails record for the named company (metadata only — does not drop the ERPNext site itself).

Related non-CRM-prefixed endpoints used in the same flow

METHOD	PATH	BODY	PURPOSE
--------	------	------	---------

POST	/api/auth/login	username, password (form-encoded)	Obtain the bearer access token used on every call above
PUT	/api/bot/{bot}/settings	{ "enable_crm": true, "analytics": {...} }	Turns on the is_crm flag that gates the whole flow

4. kairon_connector inbound webhook gateway API

Runs on each provisioned tenant site (installed by kairon_connector), not on the Kairon server. This is the channel a real Kairon bot conversation pipeline would use to push lead/conversation events into that tenant's Frappe CRM.

POST /api/method/kairon_connector.api.v1.webhook.receive_event

allow_guest=True — authenticated entirely by HMAC signature, not a session/token.

HEADER	PURPOSE
X-Kairon-Signature	sha256=<hex hmac of raw body, keyed by the tenant's lead_webhook_secret>
X-Kairon-Timestamp	Epoch seconds; request rejected if more than ±300s from server time
X-Kairon-Event-ID	Used for transport-level dedup (Redis, 24h TTL)

Request body (JSON envelope v1.0):

```
{
  "event_type": "lead.qualified",
  "event_version": "1.0",
  "event_id": "evt_e575eafaf98410c873f4b669734a406",
  "correlation_id": "corr_1234567890",
  "timestamp": "2026-09-13T14:12:47.000000Z",
  "payload": {
    "bot_id": "6aa6a5cad020d90cea7499e6",
    "conversation_id": "conv_uat_verification_001",
    "lead_data": {
      "name": "Priya Sharma",
      "email": "priya.sharma@example.com",
      "phone": "+91-9876543210",
      "company": "Sharma Textiles",
      "qualification_score": 85,
      "intent": "Enterprise CRM subscription",
      "budget": "5-10L INR",
      "industry": "Textiles"
    },
    "summary": "Prospect qualified for CRM demo.",
    "transcript": "Customer: ...\nBot: ..."
  }
}
```

HTTP	CODE	MEANING
202	queued	Accepted, enqueued to the async RQ worker for routing

200	KRN003	duplicate_ignored — same event_id seen within 24h
401	KRN001	Invalid HMAC signature
401	KRN002	Timestamp outside the replay window
400	KRN004 / KRN006	Malformed envelope / unsupported event_version

Registered event types → handlers: `lead.created`, `lead.qualified` → `LeadHandler` (creates/updates a CRM Lead + Communication + FCRM Note); `conversation.message.received`, `crm.reply.received` → `ConversationMessageHandler`.

GET `/api/method/kairon_connector.api.v1.health.get_health`

Readiness probe — checks DB (SELECT 1), Redis, queue status, and configuration. Zero secret exposure. Returns 200 (healthy/degraded) or 503 (unhealthy).

5. Outbound ERPNext APIs Kairon calls internally

Called by ERPNextClient (kairon/crm/services/erpnext_client.py) against the tenant's own site during provisioning and ongoing management. Session-authenticated as Administrator during provisioning; token-authenticated as the dedicated kairon-crm@<site> integration user afterward.

METHOD & PATH	USED FOR
POST /api/method/login	Session login as Administrator
POST /api/method/frappe.core.doctype.user.user.generate_keys	Issue API key/secret for the integration user
POST /api/resource/Company	Create the ERPNext Company (Tier 2 only)
POST /api/resource/User / PUT /api/resource/User/{name}	Create/update the integration user and the tenant owner's user
POST /api/resource/User Permission	Set the tenant owner's default Company (Tier 2 only)
POST /api/resource/Webhook	Register the ERPNext → Kairon User Invitation webhook
PUT /api/resource/Kairon Settings/Kairon Settings	Configure the Kairon → ERPNext lead-sync webhook secret (Single DocType — note the doubled name segment)
POST /api/method/frappe.core.doctype.user.user.reset_password	Explicit fallback trigger for the set-new-password email
POST /api/resource/Module Profile	Create/update the per-tenant module-visibility profile
GET/POST /api/resource/Workspace	Read/toggle workspace visibility per selected module

6. MongoDB collections & schema

Database: conversations (from DATABASE_URL). Collection names are MongoEngine's default snake-case-per-class-name conversion — confirmed empirically against the live database.

c_r_m_client_details ← model **CRMClientDetails**

One document per (bot, company_name). Unique index on (bot, company_name). The single source of truth for provisioning state, owned entirely by Kairon (Frappe doesn't know these states exist).

FIELD	TYPE	NOTES
company_name, abbr, default_currency, country	String	Required; the tenant-info API contract fields
bot, user	String	Owning bot ID and the Kairon user who triggered onboarding
onboarding_status	String enum	See state list below
tier	Int	1 (standalone) or 2 (full suite)
installed_apps	[String]	e.g. ["crm"] or ["erpnext"]
selected_modules	[String]	Business module keys chosen at onboarding
site_name, project_id	String	{clean_company_name}.localhost
postgres_host/port, db_name, db_user, db_password	String/Int	db_password encrypted (Fernet, via Utility.encrypt_message)
lock, lock_timestamp, provisioning_id	Bool/Date/String	Atomic-lock idempotency fields; stale locks (>2h) auto-recoverable
attempt_number, last_retry, last_error	Int/Date/String	Retry bookkeeping
erpnext_site, erpnext_owner_email, erpnext_user, erpnext_password, erpnext_company, erpnext_roles	mixed	erpnext_password encrypted; only set when SMTP is disabled (temp-password path)
kairon_integration_user, kairon_api_key, kairon_api_secret	String	Dedicated kairon-crm@<site> account; key/secret encrypted
webhook_secret	String (encrypted)	ERPNext → Kairon invitation-webhook HMAC key
lead_webhook_secret	String (encrypted)	Kairon → kairon_connector lead-sync HMAC key (new field, added this session)
module_profile_name	String	ERPNext Module Profile doc name for this site

timestamp, workflow_started_at,	Date
workflow_completed_at	

onboarding_status values: PENDING → PROJECT_CREATED → BENCH_RUNNING → SITE_CREATED → SITE_HEALTHY → COMPANY_CREATED → USER_CREATED → COMPLETED, plus UPGRADING_TO_TIER_2 and six FAILED_* states (one per stage). Transitions are validated against an explicit allow-list; invalid transitions are logged and ignored rather than corrupting state.

c_r_m_invitation ← model **CRMInvitation**

Lightweight audit trail per invited user — the authoritative state always lives in ERPNext's own `User Invitation` DocType; this collection never duplicates it, only tracks Kairon-side metadata.

FIELD	TYPE	NOTES
bot, user, site_name, company, email, roles	mixed	
invitation_status	String enum	Pending, Accepted, Expired, Cancelled, CompanyAssigned
erpnext_invitation_id	String	Links back to ERPNext's own record for cancel/resend
invited_at, accepted_at, cancelled_at, company_assigned_at, last_error	Date/String	

bot_settings ← model **BotSettings** (existing, shared)

The `is_crm` gate itself is one boolean field on Kairon's existing per-bot settings document — not a new collection.

FIELD	TYPE	NOTES
enable_crm	Bool (default false)	Read via <code>MongoProcessor.is_crm_enabled(bot)</code> ; onboarding is refused with <code>AppException</code> if false

7. Live verification evidence

All results below are from real calls against a running stack (Frappe/ERPNext in Docker, Kairon API on :8001, MongoDB, Mailpit) — not unit tests with mocks.

Tier 1 — Kairon Demo Restaurant (CRM-only)

CHECK	RESULT
POST /crm/onboard → final status	COMPLETED in a single clean pass (~68s)
Apps installed on site	frappe, crm, kairon_connector
CRM workspace visibility	public: 1 (visible, not hidden)
Welcome email	Delivered to Mailpit; /update-password?key=. . . link extracted and opened
Set-password screen	Confirmed via browser screenshot — real "Set Password" form rendered
Post-password login	Auto-logged in, landed on /crm/leads/view/list — the actual Frappe CRM app UI

Tier 2 — Kairon Tier2 Regression Check (ERPNext/POS)

CHECK	RESULT
POST /crm/onboard → final status	COMPLETED in a single clean pass (~3m31s)
Apps installed on site	frappe, erpnext, kairon_connector
ERPNext Company record	Exists — <code>frappe.db.exists('Company', 'Kairon Tier2 Regression Check')</code> → true
Tenant owner User record	Exists (before the fix: did not exist at all — only Administrator)
Email Queue	2 messages, both Sent (before the fix: 0)
CRMClientDetails metadata	lock: false, kairon_api_key/lead_webhook_secret/module_profile_name all populated

CRM lead-sync webhook — end to end

CHECK	RESULT
Secret retrieval	<code>CRMPProcessor.get_lead_webhook_secret(bot)</code> returned the real, auto-provisioned per-tenant secret

Signed lead.qualified event	202 queued → Kairon Webhook Log status Success
Resulting CRM Lead	Created with all custom fields correctly mapped (score 85, intent, budget, industry, conversation_id, bot_id)
Activity timeline	Communication (transcript) + FCRM Note both created and linked
Replay same event_id	200 KRN003 duplicate_ignored — idempotency confirmed
Wrong HMAC secret	401 KRN001 — signature rejection confirmed (and no "default_secret" fallback, per fix #2)

Regression check

Existing unit suite (tests/unit_test/crm_test.py): **36 passed, 5 failed** — identical failing tests on the unmodified pre-session code (verified via git stash), so none of the 5 are new regressions from this session's changes.

8. Known limitations / follow-ups

- **Duplicate welcome emails** on runs where the native Frappe hook does fire (see §2, fix 8) — cosmetic, both links work.
- **Resumability beyond one continuous request** is verified for retry-from-FAILED_USER only; other failure/resume combinations across separate API calls were not each individually exercised.
- **Duplicated doctype directory tree** in apps/kairon_connector (three physical copies of some DocType JSON/py files, e.g. core/doctype/kairon_settings/ and a doubly-nested kairon_connector/kairon_connector/kairon_connector/doctype/). Confirmed only the canonical apps/kairon_connector/kairon_connector/doctype/ copy is actually loaded (verified via live `frappe.get_meta(...).module`) — the others are dead weight, not a functional bug, and a `before_migrate` hook (`sync_module_tree`) already exists specifically to guard against this. Left untouched.
- **Non-crm module role/permission lists** (helpdesk, hr, selling, accounts, pos in `provisioning_matrix.yaml`) were audited by inspection only — only the `crm` entry's role list was proven broken by live evidence and fixed. The others appear to reference real ERPNext role names but were not each individually exercised end-to-end this session (Tier 2/POS *was* exercised and passed).
- A pre-existing, uncommitted change to `provisioning_matrix.yaml`'s `pos` section (adding a "Restaurant POS" workspace/home_page) was present before this session started and was left untouched — it is not part of this session's diff.

Generated from a live working session against the Kairon + Frappe/ERPNext stack described on the cover page. All code changes referenced above are in the working tree of the respective repositories, uncommitted, pending review.